

Deep Learning - Part II

✓6. GAN'S

✓7. Encoder-Decoder Models

✓8. Attention Models

✓9. Transformers & BERT

✓10. Image Segmentation

11. Object Detection (2 videos)

Part 1	Part 2
--------	--------

12. GPT-1, 2, 3 Models

13. OpenCV using Python

14. Interview Questions

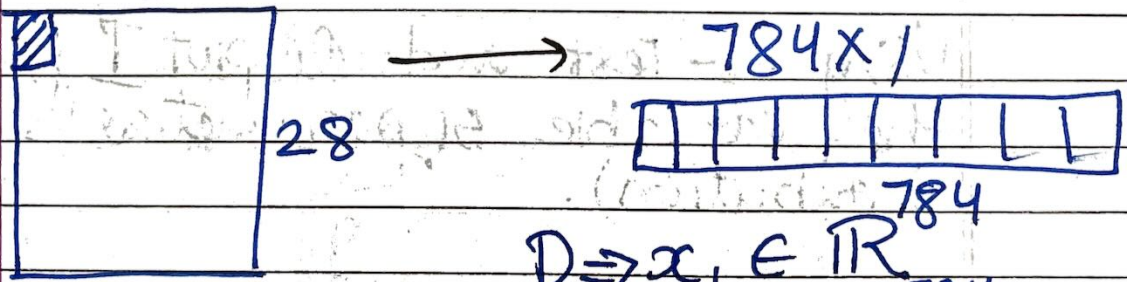
15. Live Sessions

ch6 Generative Adversarial Networks

- 1 First Principal of GANs
- 2 Overview of GANs
- 3 Keras Code

Given a bunch of MNIST image dataset D , I want to generate D' which are new MNIST dataset and these images are very similar to the train images but not same.

Each pixel is a grayscale value.



$$D \Rightarrow \begin{aligned} x_1 &\in \mathbb{R}^{784} \\ x_2 &\in \mathbb{R}^{784} \\ x_3 &\in \mathbb{R}^{784} \\ &\vdots \\ x_n &\in \mathbb{R}^{784} \end{aligned}$$

28x28
images

$$D' = \begin{aligned} x'_1 &\in \mathbb{R}^{784} \\ x'_2 &\in \mathbb{R}^{784} \\ &\vdots \\ x'_n &\in \mathbb{R}^{784} \end{aligned}$$

$$x_n \in \mathbb{R}^{784}$$

First Principle Approach (Simple Task)

1. Generate scalars (numbers or 1-D) which is D' (heights).

(Height) D = Set of heights of students.

$$= \{h_1, h_2, \dots, h_n\}$$

D' = Set of heights of students similar to D .

$$= \{h'_1, h'_2, h'_3, \dots, h'_n\}$$

2. $\rightarrow$ First generate PDF. and plot and determine the distribution.

Using KS-Test and QQ plot I figured the variable θ (parameters of distribution).

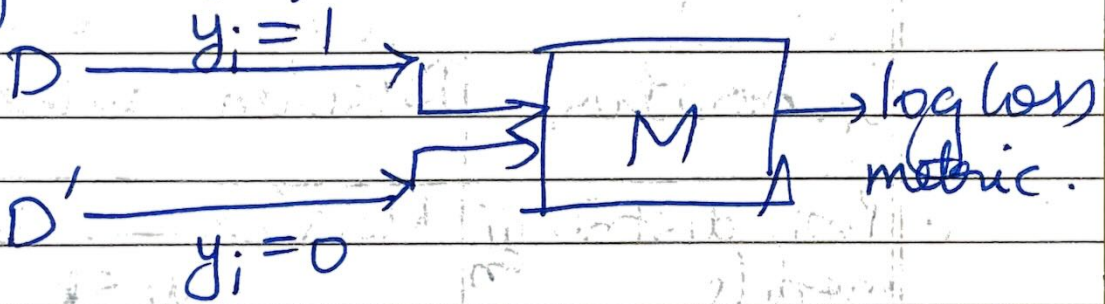
We will use Random numbers using the parameter values.

Similar in generating new sample mean that their distributions should be same.

3. Measure how similar D and D' are?

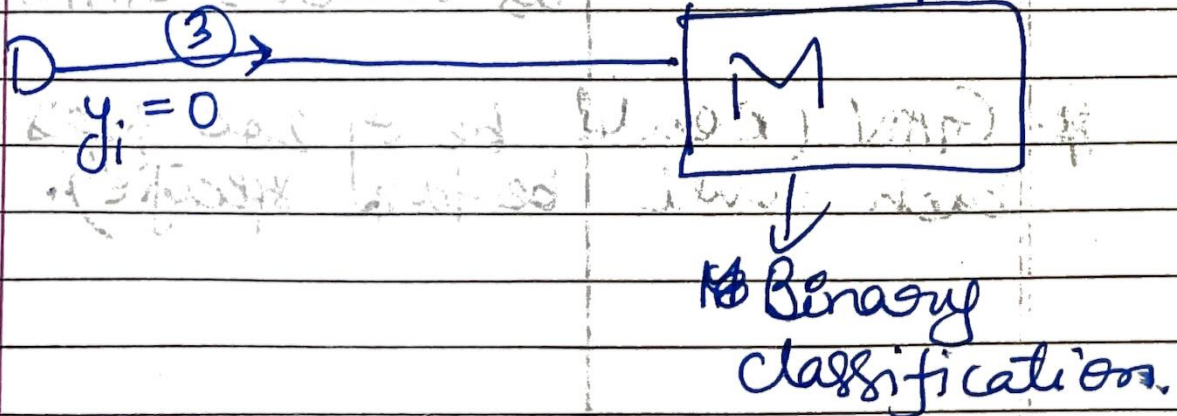
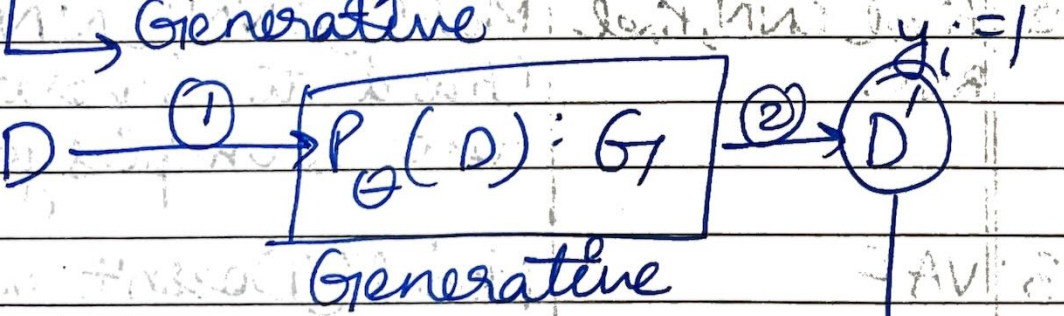
→
Metric
• KL divergence
• Jensen-Shannon distance.

→ Model based approach can also be used.
If model performs well its ^{binary} ~~categorical~~ cross-entropy (log loss) would be low. And it will be able to distinguish b/w D and D' . And if it performs bad, then D will be $= D'$.



GAN

↳ Discriminative
↳ Generative



Main Topic → MNIST data replication

In extreme high dimensions, KL-divergence or Jensen-Shannon distance metrics don't work and that is why model based strategy is used to judge the ^{similarity} of a ~~model~~ distribution! _{See}

DNN/CNN work as a generator.

Generators	Discriminator
1. Prob distribution models	Metrics → Doesn't work for high dimensional data vectors.
2. Pixel CNN, Pixel RNN.	Model (DNN/CNN) (Hard to generate new samples from).
3. VAE	Model (Doesn't work as well as GANs)
4. GAN (could be of 200 types. Each could be task specific).	

Applications of GANs.

1. Domain Transfer
2. Anime Coloring
3. Super Resolution GAN (Seminal Paper).
4. Progressive GANs.

GANs is $\rightarrow$ fix D update Train G
fix G Train D.
check wikipedia for detailed explanation!

3) Deep Visual-Semantic Alignment for Generating Image Description by Andrej Karpathy, et al

V.7.1

Encoder-Decoder Models

Invented at Google Research

Discussed in LSTM chapter (Types of RNN).

Appendix for next videos :-

Encoder-Decoder why do we need RNN Attention Models

Transfer " " discussed in BERT: Bidirectional RNN chapter under Representation from Transformers.

References :- (1) Sequence to Sequence Ilya Sutskever

(2) Cho, et al Learning Phrase Rep. using RNN encoder-decoder for Statistical Machine Translation

Seq to Seq Model : (Machine Translation)

Practical: Used by Google Translate as core algo for la while Email: Auto Reply.

Image-Text.
Auto Suggestion.

Machine Translation:

$\langle x^1 x^2 x^3 x^4 x^5 \dots x^t \rangle$
 Input English
 $\downarrow M$

$y^1 y^2 y^3 y^4 y^5 \dots y^k$
 Output (Hindi)

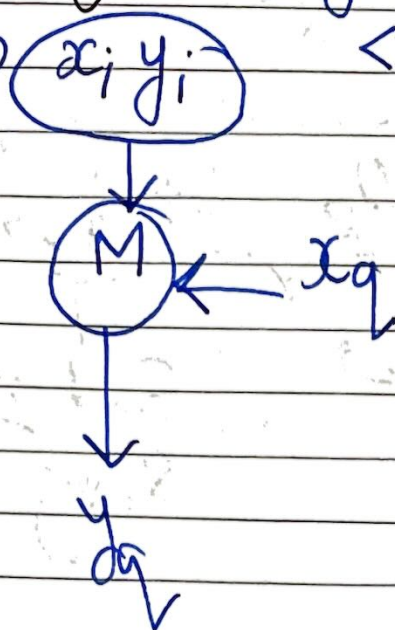
(*) t need not be equal to k .

Given x_i ^{sequence}, I want to compute y_i ^{sequence}.

Image Captioning/Description (3 Ref)

Image $\rightarrow \langle y^1, y^2, y^3, \dots, y^k \rangle$

Mathematically, $P(y^1, y^2, y^3, y^4 \dots y^K)$
Trains: $D(x_i, y_i)$ $\langle x^1, x^2, x^3, \dots x^K \rangle$

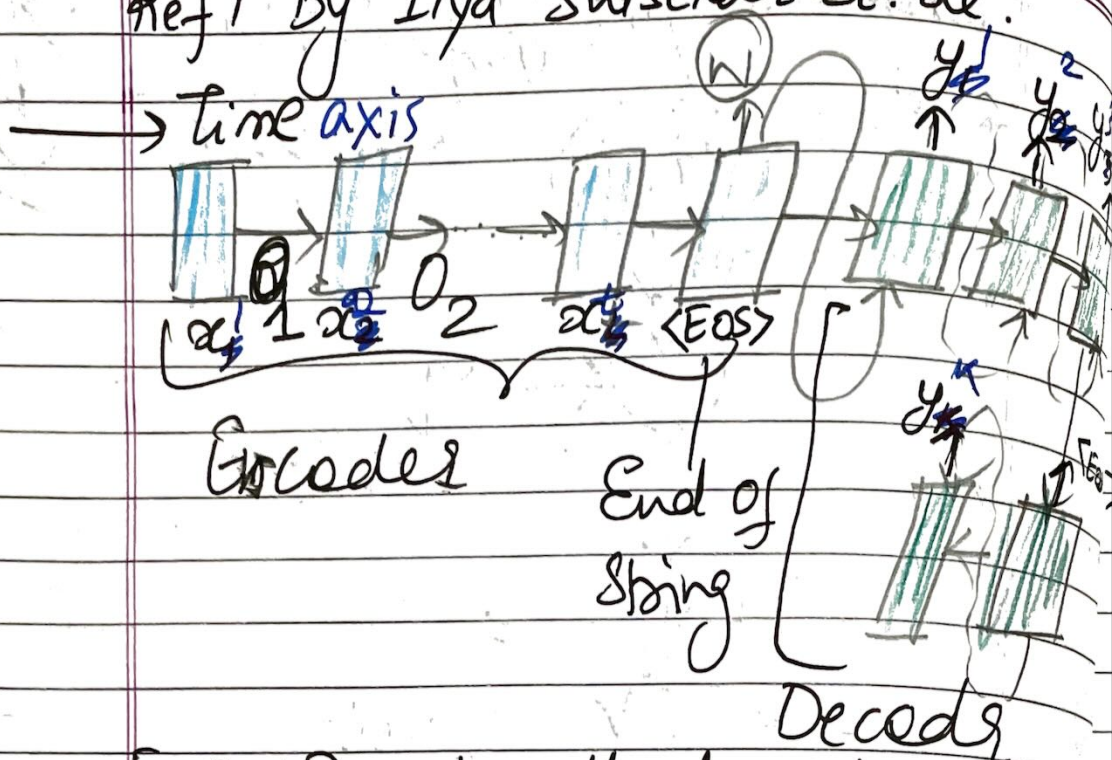


Given any probability for y given x . It can train and for any query point we can find y_q given x_q .

Optimization:

$$\hat{y} = \underset{y}{\operatorname{argmax}} P(y/x)$$

Ref 1 by Ilya Sutskever et. al.



Eos: Denotes that string has ended.

$w \rightarrow$ context vector.

$x_1, x_2, \dots, x_t$ Don't have output

w is ip to the first cell in the decoder model.

LSTM at green generates o/p

Superscript: first used in the encoder

Subscript:

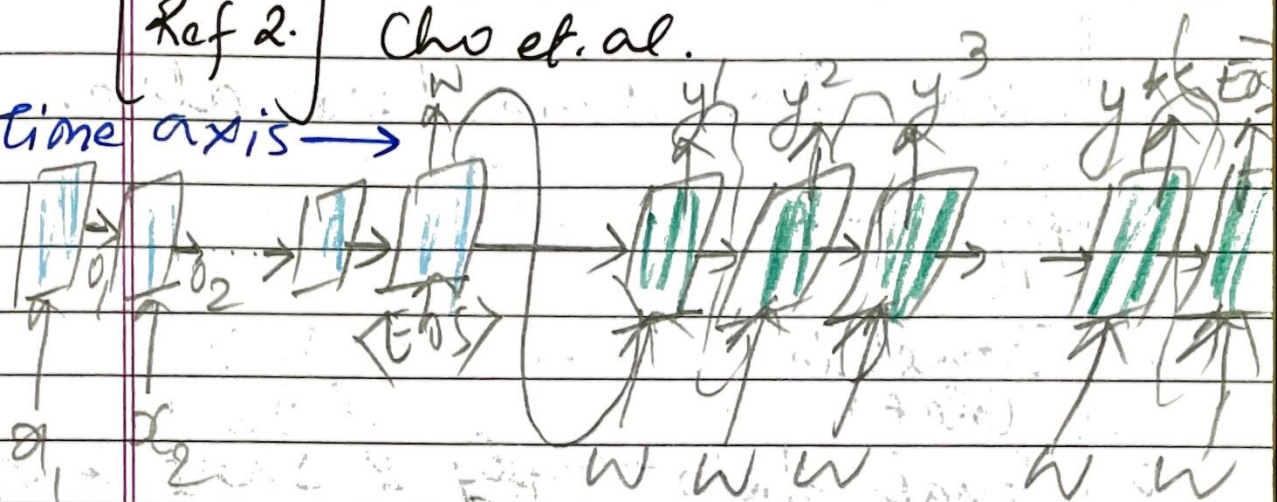
We create loss function in LSTM (maybe softmax).

loss going $\downarrow$ means loss is small

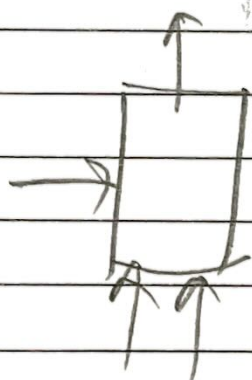
Backpropagation in time.

[Ref 2.] Cho et. al.

time axis $\rightarrow$



Feeding w to LSTM can't happen. So Cho et al modified the LSTM to Simple LSTM.



Modified LSTM unit.

Note Context Vector w is the output when we given EOS as i/p.

Ref 3 Andrey Karpathy & Rishabh

Diag present in the paper

This has Image Context Vectors (w)
This is fed into the LSTM model
which is the decoder.
Encoder is CNN.

Same Time axis here also.

Applications

1. Translate
(Ref)
2. Email Auto-reply & Smart Compose
(Ref)
3. Code errors
(Ref)
4. Image descriptions
(Ref)

Problem with Simple Seq 2 seq Model.

If you have a v. lengthy sentence, the (u) context vec. cannot capture the essence of x .

Concept of Attention Model is to solve the problem of how humans translate.

Now it's better $\therefore$ of Attention Models.

Keras Code Discussed.

8.1 Attention Models in Deep Learning

— why do we need them?

Popular in 2015 to 2017

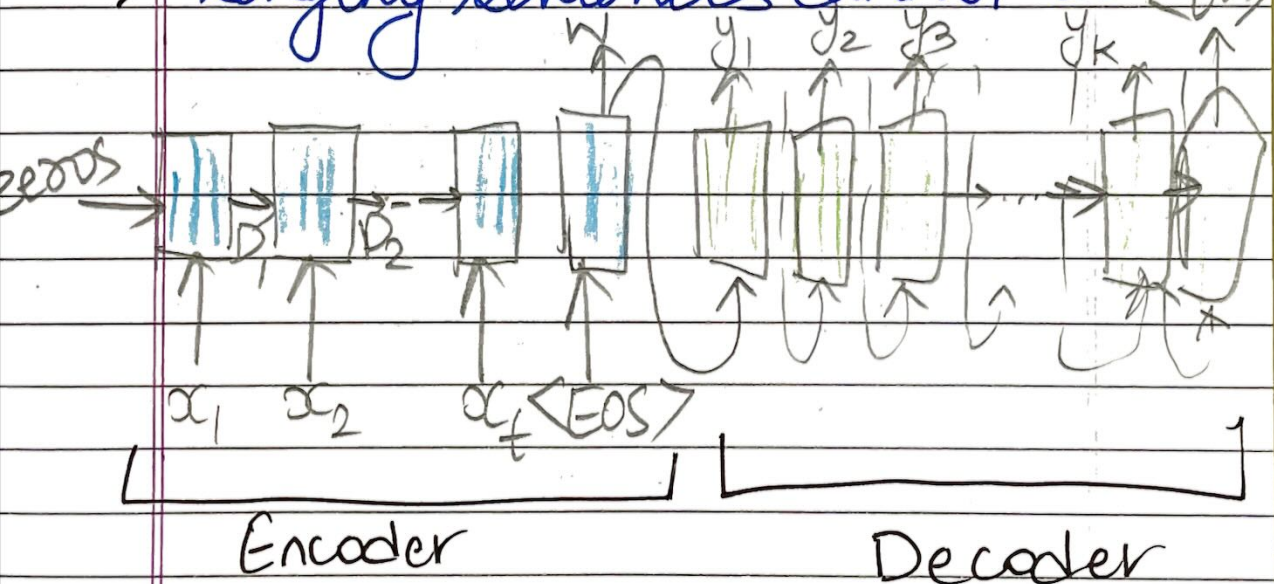
Transformers became v. popular

BERT became famous in 2018.
It is an example of a Transformer.

— One-line code:

Problems with Simple Seq2Seq model:

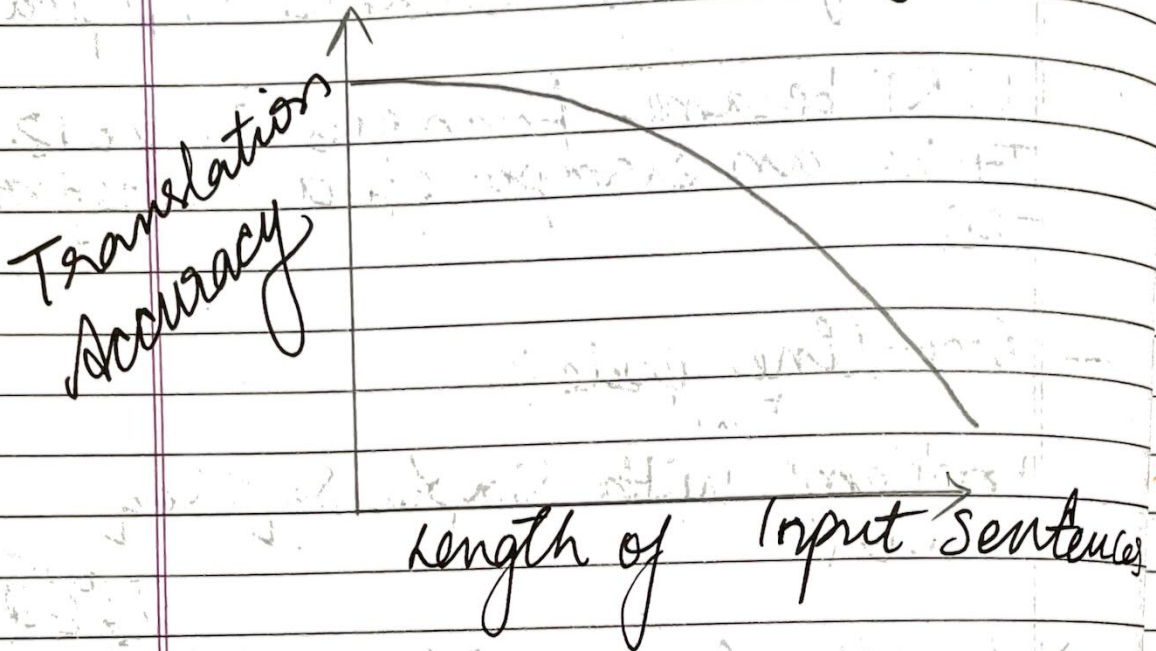
→ Lengthy sentences cannot be ^{trans}lated ^{lated} ^{EOS}



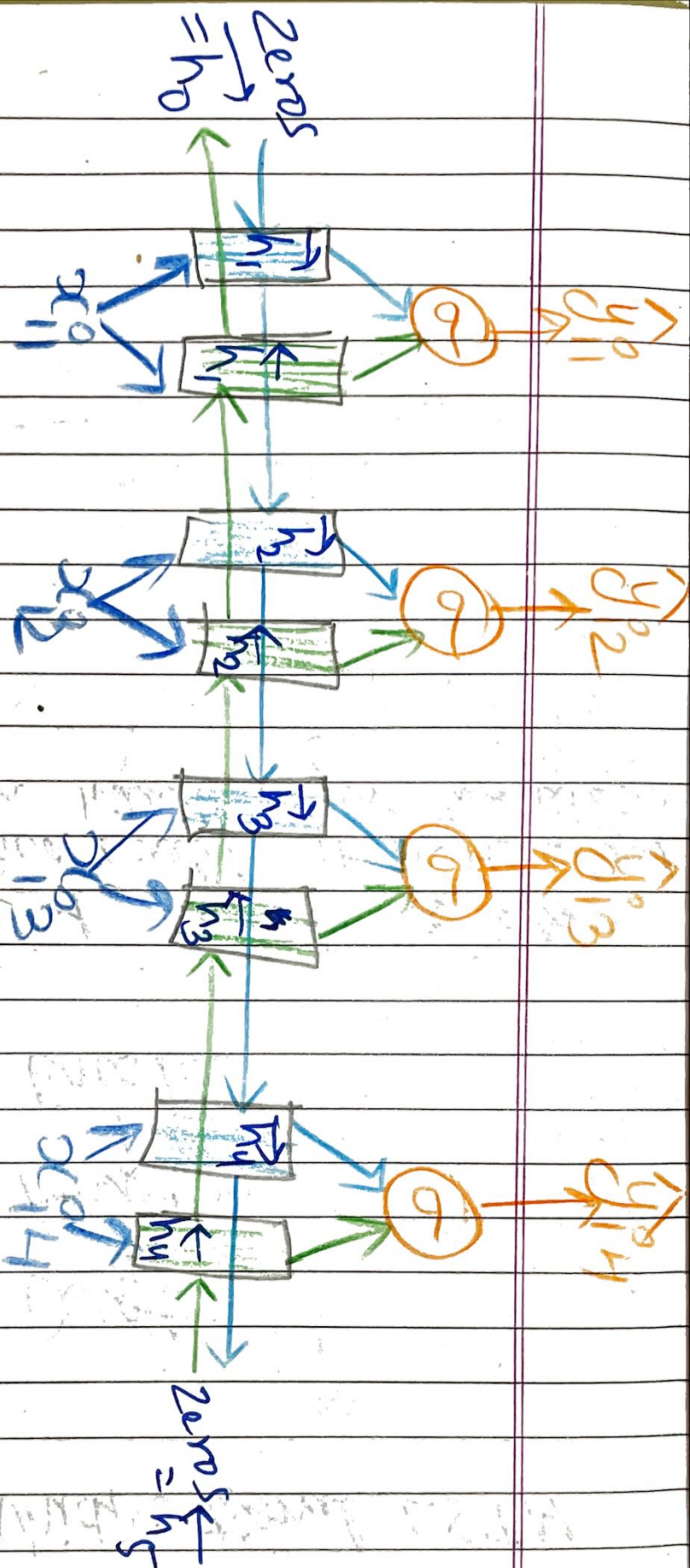
Q Why we call it Attention?

Ans Because humans focus on certain words and translate it on and then focus on other words and so on.

Translation Accuracy:-



Bidirectional RNN



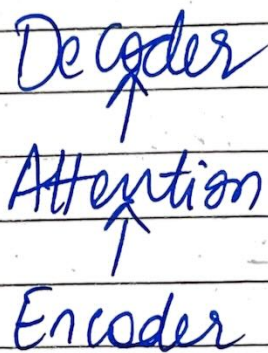
$x_0 = i^{\text{th}}$ i/p sentence
 $y_0 = i^{\text{th}}$ o/p "

$D = \{x_i, y_i\}_{i=1}^n$
 $w_t = \{h_t, h_t\}$

Attention Models

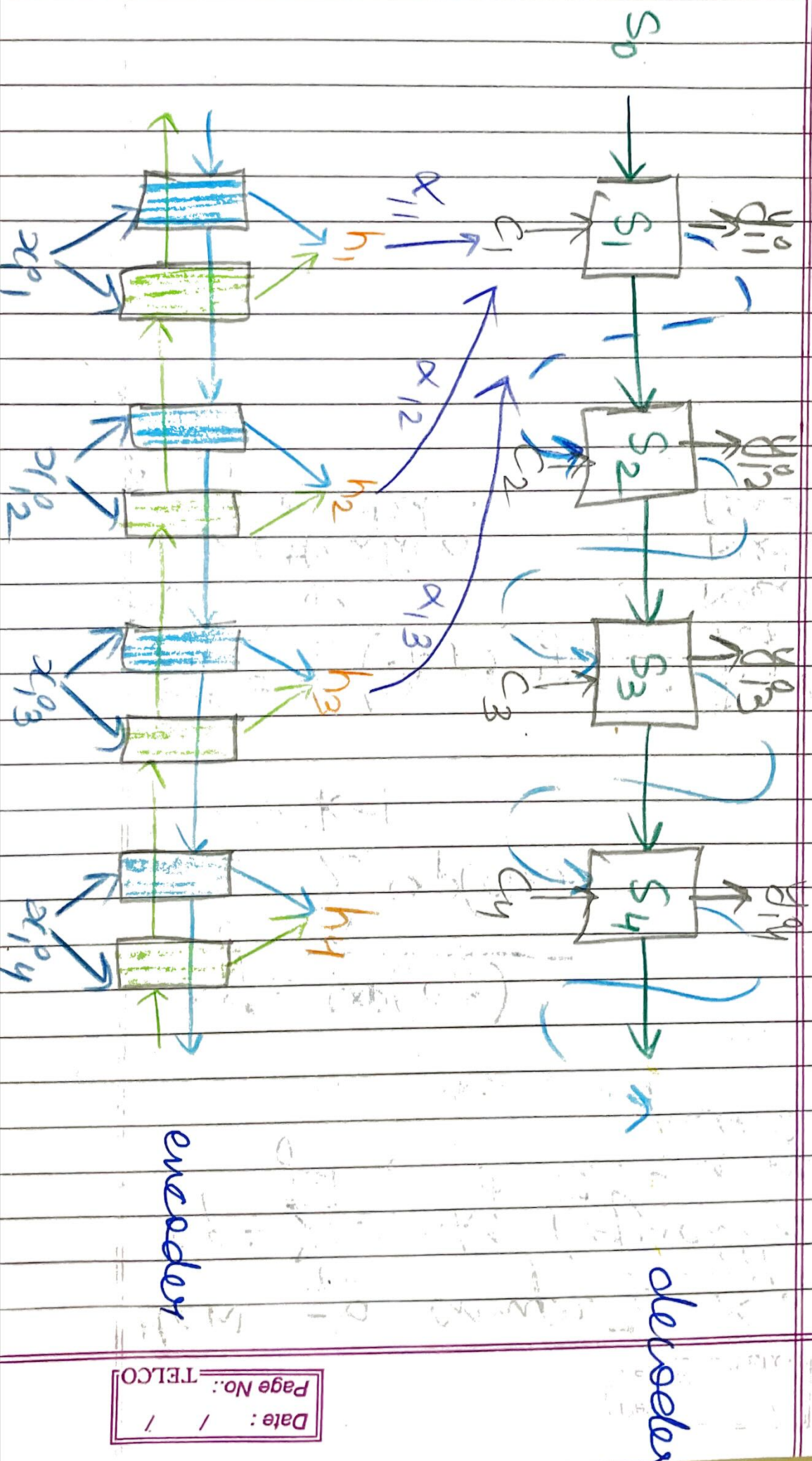
Encoder : Bidirectional LSTM

Decoder : Unidirectional LSTM



This type of models is also called prior model. It is developed using Human Intuition for Translation.

Attention Model



How to compute α_{ij}^o 's

$$C_i^o = \sum_{j=1}^{T_x} \alpha_{ij}^o h_j^o \quad (\text{equation 5 in the research paper})$$

$$\alpha_{ij}^o = \frac{\exp(e_{ij}^o)}{\sum_{k=1}^{T_x} \exp(e_{ik}^o)}$$

$$e_{ij}^o = a(s_{i-1}, h_j) \quad \text{Feed Forward Neural Network (small)}$$

≥ 0 ↓ attention model.

$$\sum_j \alpha_{ij} = 1$$

Drawback:

$$\text{Time-complexity} = O(k_1 \cdot k_2)$$

It's not a
major drawback

length
of o/p
Sentence

length of i/p
Sentence

Online implementation.
References shared.

V.8.2 Transformers and BERT

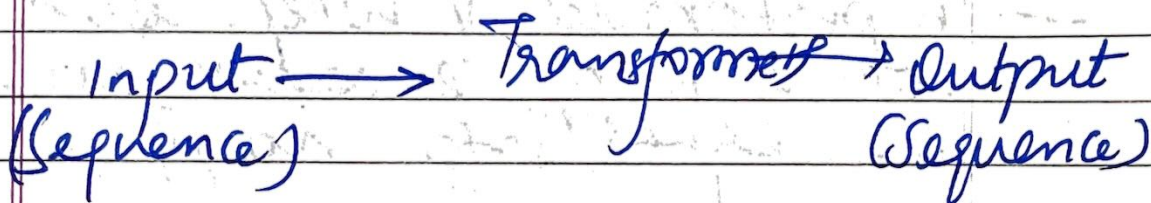
(NLP technique)

Transformers

BERT - Bidirectional encoder representation from Transformers

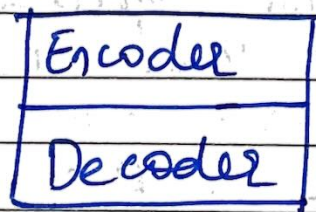
Text & Images taken from Salammar blog.

- A transformer is an advanced seq-to-seq model.



Let's consider a machine translation problem to convert a seq. from French to English.

Transformer ⇒



Imp Spl. type of model which are not RNN model but Attention models

Original Transformer paper:

⇒ Attention is all you need.

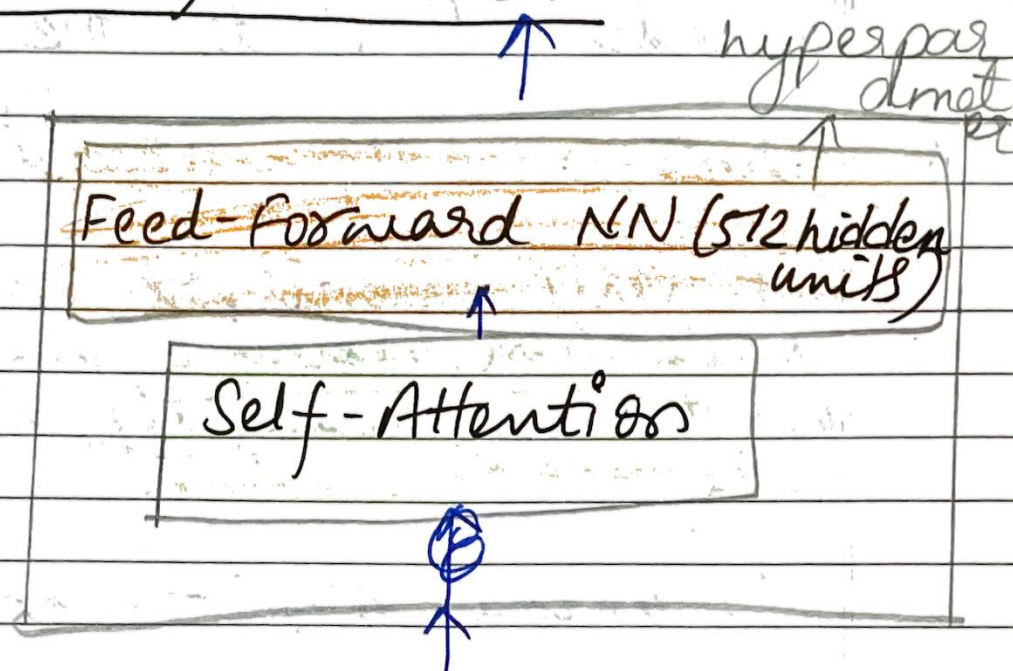
* Block of Encoder = 6 seq of Encoder

& " " Decoder = 6 seq of Decoder

as per the original paper.

- 6 performed v. well. Also both no. of encoders must be equal to no. of decoders.
- Each of the encoders are identical. So is decoder.
- I/p Sentence is fed to the 1st encoder whose o/p goes to 2nd " so on and so forth.
- The o/p of the last encoder goes to all the decoders.

Block of an encoder :-



- FFNN $\rightarrow$ Relu based hidden units.
- Self-Attention = ?

Q How is an i/p given to an encoder?

$\rightarrow$ convert words to ~~words~~ vector using W2V.

W_1, W_2, W_3 ~~are~~ All different words are encoded with all different 512 encoded vectors.

- Different from RNN/So all the vectors are fed to the encoder all at one time.

Q what is Self-Attention?

- It very ingeniously says certain words on certain words which are sensefully meant. like

The animal didn't cross the street because it was too tired.

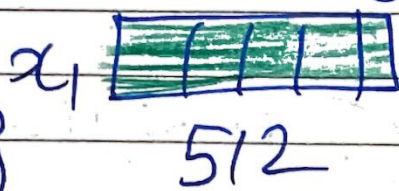
→ Maths behind Self Attention's intelligent interpretation mechanism

Don't give word but ~~word~~ vector of 512-D

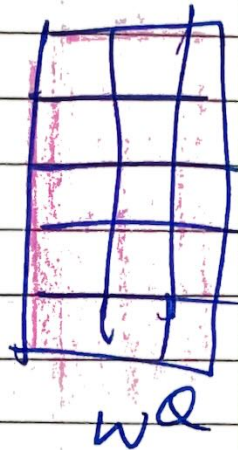
Step - 1

I/P Thinking Machines

~~Queries~~
Embedding



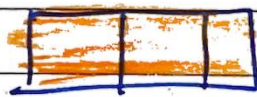
Queries



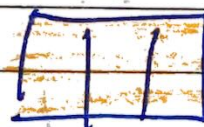
$q_1 = X_p W^Q$ q_1 64 = d_k

q_2

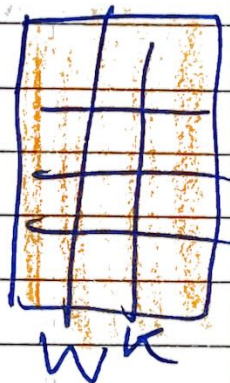
Keys



K_1 64



K_2 64



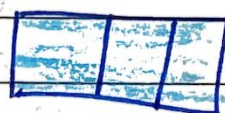
$K_1 = X_p W^K$

Values

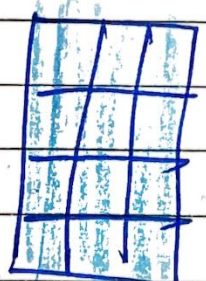


64

v_1



v_2



w^V

$V_1 = X_p W^V$

Training
matrix

Step 2: We compute score.

Query (dot product) keys

$$q_p \cdot k_p$$

Suppose:

$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$
-----------------------	----------------------

q_1 is multiplied $\therefore$ we want q_1 to generate z_1 .

Step 3: Divide by $8(\sqrt{d_k})$

Q why 8? (Normalization step)

$\therefore$ 8 is sq. root of 64.
and $d_k = 64$.

$\therefore$ whole model is using gradient descent model.

Then apply softmax_k^{func.} on the result.

Step 4.

- Softmax ensures that the value we get is proportional to the Division result.

Step 5 Take the Softmax, multiply it with the values.

ie, multiply softmax with v_1 .

Step 6 Sum up all of that

$$Z_1 = \text{sum}(v_1, \text{softmax}, v_2, \text{softmax})$$

32:20

Multiheaded Attention

1 set of Attention head: w_Q, w_K, w_V
The first head didn't tell that tired was also imp.

Since single-headed attention didn't work, so authors did hyperparameter tuning which resulted in 8 set of matrices. It increases the representational power of your model

Now, how to combine the results of all attention models? We do concatenate (Z 's). Multiply with another weight w_0 .

Look at the summary in notes.

- You measure the loss, backpropagate the whole loss and adjust all weights of the model and that's how gradient descent works.

Positional Encoding

We have not encoded the order of words.

In RNN, we do train the order of words.

55:27.

Positional vector is added to the embedding. In paper, it is described how to use positional argument.

In every encoder step, we normalize so that gradients don't explode.

Encoder-Decoder Attention is i/p from encoder's output.

All the words are added encoded at the same time and decoder has a time based phenomenon.

BERT - Simpler concept.

BERT is for transfer learning in NLP.

- Text classification.
- Sentiment Analysis
-

BERT is not a replacement for Transformers. BERT is a simpler version of a transformer. - risky

Q Given a sentence, can I get a vector?

- We use BERT for that.

→ BERT-base (smaller) → FFN(768) & Attention heads(12)

→ BERT-large (larger) → FFN(1024) & Attention heads(16)

BERT beats humans that beats them in Q/A dataset.

They are only encoder stacks. Also called Transformer Blocks.

Forward-only → Flow in one direction

Date: / /

Page No.: /

TELCO

CLS - Classification

Each op is a 768-dim vector in BERT

Q How is BERT trained?

- It is a bi-directional model. So all the words have an impact on every other word.

Q i/p data?

- Masked language model.

* Within the BERT, we randomly mask of tokens.

Task specific models

- B - sentence pair classification task
- E - single sentence tagging task
- R - Q/A tasks
- T - single sentence classification task

BERT as a feature extraction system:

- Generate contextualized embeddings
- What is a contextualized embedding for "Help" in that context?

DistilBERT → pretrained model. Pass words through this and use the word vectors through logistic regression.